

A calculator that computes mathematical results, keeps a running history of every calculation in the session and lets user recall previous results and use them in new calculations.

What you'll learn :

1. Structuring functions properly
2. Storing data in lists during a session
3. Input Validation (what happens if user types "abc" instead of a number?)
4. Infinite loops with clean exit conditions

Relevance :

Input validation and error handling are critical in production ML code and financial applications.

Project Philosophy :

Making a basic calculator that runs in the terminal without a GUI. The user types in the calculation in the working command line and presses enter to get the result on the next command line and next to next command line becomes the active command line where user can type in a new calculation. Meanwhile, the calculation history is stored and the user can run command to use a previous result.

User Workflow :

1. Open terminal and run the program
2. Type a calculation 7×8 in the > command line
3. Clicks Enter
4. Gets result 56 on the next line
5. User now redirected on third line for new active calculation.

```
> 7*8
> 56
> |
```

6. User can use previous results by using \r1 , \r2 , etc to use in new calculations

```
> \r1 + 3
> 59
```

PseudoCode :

1. Start
2. An infinite loop starts which takes user input & prints it as program can do calculations by itself
3. Store output in an array "result" at every iteration

Progress :

1. wrote the input & print function but program printed 4+5 itself when given 4+5 and not 9. Found out about the eval() function.

```
x = input("Type to calculate:") print(x)`
```

2. implemented while loop to keep asking calculations , keep printing evaluation and print result in stored list but list is not persistent and it shows only the latest result and not previous ones. fix : the line results = [] should sit outside the while loop as it is getting overwritten by ' ' everytime loop is initiated.

```
results = []
while True:
    x = input("Enter to Calculate")
    print(eval(x))
    results.append(x)
    print(results)
```

3. while adding stop command , I needed to figure out a way to break out of the while loop to end the program. if the user types in stop the program should end. So using break statement if input = stop then break. operator can't be used on string "stop" so I declared y = "stop" outside the loop and added if x==y: break

```
y = "stop"
results = []
while True:
    x = input("Enter to Calculate")
    if x==y:
        break
    print(eval(x))
    results.append(x)
    print(results)
```

4. added a specific command 'rone' which prints the first element of the list (i.e the first result). this was the most difficult out of previous ones. when i first defined a = "rone" out of while and checked if x = a then print(results[0]) , it came out as error because next lines were being carried out and x = rone was going in eval(x) causing error because eval() only takes integer values. So I had to initiate the last three lines only when the input was not a command like stop or rone. so i used if condition x!=a and x!=y then indented lower lines to it. this logic was important because i had written x != y or a: which is wrong because it

would always be true as I have already declared a = rone so a is always true and so value of y or a would be true.

```
y = "stop"
a = "rone"
results = []
while True:
    x = input("Enter to calculate")
    if x == y:
        break
    if x == a:
        print(results[0])
    if x!=y and x!=a:
        print(eval(x))
        results.append(eval(x))
        print(results)
```

5. Now to add : generalizing command like rone, rtwo, rthree so that corresponding element is printed and I don't have to declare all rone, rtwo , etc. First of all I didn't know how to generalize any input taken from the user to pass as an index to the list of results. I wanted r0, r1, r2, etc if typed as a command to print the corresponding element in the results list. So I only need the number part of the input. As the input is a string named 'x' with first letter r and everything else the number i can do that by x[1:] so that all elements including that at index 1 of the string are taken (i.e everything after 'r'). Now i needed to convert that string into an integer to be able to pass it as an index so used int(x[1:]) and then we can use this value as an index in results ; results[int(x[1:])] and print it whenever input 'x' starts with x. for this condition I had to use the .startswith method. I used if x.startswith('r') then print(results[int(x[1:]))). So I also had to change the exclusion condition also for the eval function part , removed x!=a and used not x.startswith('r').

```
y = "stop"
results = []
while True:
    x = input("Enter to calculate")
    if x == y:
        break
    if x.startswith('r'):
        print(results[int(x[1:]))
    if x!=y and not x.startswith('r'):
        print(eval(x))
        results.append(eval(x))
        print(results)
```

6. Added a guide at the start of the program out of the while loop which prints how to use the calculator and all the commands.

```
y = "stop"
results = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
      - stop -> ends program
      - rn -> prints n'th result""")
while True:
    x = input("Enter to calculate")
    if x == y:
        break
    if x.startswith('r'):
        print(results[int(x[1:])])
    if x!=y and not x.startswith('r'):
        print(eval(x))
        results.append(eval(x))
        print(results)
```

6. I drew a flowchart of the program and realized that if the first two conditions if x=y and x.startswith('r') are not true then the third condition will be evaluated. So there is no need to write another condition in the third condition (i.e x!=y and not x.startswith('r') is redundant). I can just use else because that is the same thing.

```
y = "stop"
results = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
      - stop -> ends program
      - rn -> prints n'th result""")
while True:
    x = input("Enter to calculate")
    if x == y:
        break
    if x.startswith('r'):
        print(results[int(x[1:])])
    else:
        print(eval(x))
        results.append(eval(x))
        print(results)
```

7. Now I want to let the user use results to calculate further , like using r2 right now prints the third result but using r2+2 doesn't make a new calculation adding 2 to the third result. But if

$x = r2+2$ then `int(r2+2)` would be invalid because of the + sign $r2+2$ is not a valid integer. Like "32" can be converted to 32 by using `int()` but because of + sign there will be error. So the point is how can I separate $r2$ from 2 and also get `results[2]` instead of 2 from $r2$ itself. So I need to figure out how to transform $r2+2$ into `results[2]+2` and separate it as well. At first I thought of using `split(+)` but so that it would give me $r2$ and 2 separately but I would lose the + operator which I need in `eval()`. I can use `x.replace("r0","str(results[0]))` to replace $r0$ in the input with `results[0]` using `str()` on it as `.replace` takes strings only. So that $r0$ is converted into `results[0]` which can give number at index 0 and be used in `eval`. Now I can transform $r0+2$ into $10+2$ (assuming number 10 at index 0 in results list). Now how do I generalize this is the question.

I can generalize it by: let's consider $r10+2$ then I only want 10 from this and put it in `results[10]` so I will run a while loop after which will stop when it encounters +. so `i=1`

`while x[1].isdigit():`

as `i` increases it gives 1, 0 etc and `isdigit(1)` gives true so loop continues but ends when `i = 3` as it is + and `isdigit(+)` returns false.

we want the 10 so I will take slice from the start till where loop ends, here loop ends at `i = 3` which is + so if I slice at `x[1:i]` it gives = 10 as `i = 3`. so now I have 10 which I will pass in results as `results[x[1:i]]`, and replace $r10$ with `results[x[1:i]]` so now I can use `x.replace("r10","results[x[1:i]]")` but $r10$ from `x` can be written as `x[0:i]` so we write `x.replace("x[0:i]","results[x[1:i]]")` ``

```
i = 1
while x[i].isdigit():
    i+=1
b = x.replace(x[0:i],str(results[int(x[1:i])]))
eval(b)
```

I should replace "`x[0:i]`" because it acts like a literal string for `.replace` to find in $r10+2$ which it won't find. It should be a variable `x[0:i]`. Also `x[1:i]` will return a string "10" and `results["10"]` will give error as indexing can't be done with strings so I need to convert it using `int`. Also `b` will be something like `b = "5+2"` which can then be passed as `eval(b)`. B

But what if user types just $r10$ then what will `x[3].isdigit()` evaluate to? it will give an indexing error. So we need to check the string length and raise `i` uptill that length only. if there is an operator in the middle then the loop will break and give a result for $r10+2$ but if there is no operator then the loop should break by `i` not increasing over the string length.

```
i = 1
while x[i].isdigit() and i < len(x)
    i+=1
```

```
b = x.replace(x[0:i],str(results[int(x[1:i])]))
eval(b)
```

But if `x = r10` and `i` reaches 4 then it should stop but conditions are checked left to right so while will check `x[i].isdigit()` first as `x[4].isdigit()` before checking `i < len(x)` and give error immediately. Instead if we put `i < len(x)` first it will give false and next condition won't be checked because `and` is in between and both conditions would need to be true to pass.

```
i = 1
while i < len(x) and x[i].isdigit():
    i+=1
b = x.replace(x[0:i],str(results[int(x[1:i])]))
eval(b)
```

Now to actually display the evaluation I should write `print(eval(b))` and also replace the original code which was when input starts with 'r' with this code :

```
y = "stop"
results = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
      - stop -> ends program
      - rn -> prints n'th result""")
while True:
    x = input("Enter to calculate")
    if x == y:
        break
    if x.startswith('r'):
        i = 1
        while i < len(x) and x[i].isdigit():
            i+=1
        b = x.replace(x[0:i],str(results[int(x[1:i])]))
        print(eval(b))
    else:
        print(eval(x))
        results.append(eval(x))
        print(results)
```

ITWORKS!!

6. I forgot about one feature completely which was to show calculation history. Maybe I can add a command like `\history` which prints all the calculation history one below another. For that I will

have to store all user inputs as strings in a list and then print while indexing through all of them when x = history

```
y = "stop"
z = "history"
results = []
history = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
      - stop -> ends program
      - rn -> prints n'th result""")
while True:
    x = input("Enter to calculate")
    history.append(str(x))
    if x == y:
        break
    if x == z:
        print(history)
    if x.startswith('r'):
        i = 1
        while i < len(x) and x[i].isdigit():
            i+=1
            b = x.replace(x[0:i],str(results[int(x[1:i])]))
            print(eval(b))
    else:
        print(eval(x))
        results.append(eval(x))
        print(results)
```

But this prints the history in a list format, which I think is actually better than seeing it be printed line by line.

6. Right now, r0-2 works but 2-r0 won't work. This is because the condition has only checked for startswith(r) and indexes from the start of the string. Right now we are assuming that if x ever has r as input then it will be at the start hence having index 0. But for something like 4-r2, the r is located at index 2. So now instead of assuming r is always at index 0, I have to find where r is every time. The .index() string method will give the index of a particular character in a string. So now I have to replace 0 for r with x.index('r').

Now let's consider I have input x = 4-r2.

then x doesn't start with r but neither ends with it, so I have to scrap `if x.startswith('r')` and replace it with `if "r" in x`: then initially I wrote starting index for i as 1 because r was always at 0 but now r can be anywhere and its index is given by `x.index('r')` so i should start a position after it to get the 2 from r2 `i = x.index('r')+1` now length of x = 4-r2 is 3 but i

already starts at 3 so that will be no problem. Then making a new variable b and storing a new x in it by replacing r2 from it. Before we assumed it will always start with rN hence we did x[0:i] but now 0 is replaced by x.index('r') and i will remain i which will give us the number after r so first argument of replace function will be x[x.index('r'):i] now to get the number itself to put inside the index of results[] we just need to exclude r so we can do x[x.index('r')+1:i] . Also the line b = x.replace(x[x.index('r'):i],str(results[int(x[x.index('r')+1:i]))]) shouldn't be inside while or else it is repeated for each i . At the end of the while loop whatever i becomes , we want that to be put into as an argument in this line wo we add that and eval(b) outside the while loop after it ends.

putting it together we get :

```
if "r" in x:
    i = x.index('r')+1
    while i < len(x) and x[i].isdigit():
        i+=1
    b = x.replace(x[x.index('r'):i],str(results[int(x[x.index('r')+1:i]))])
    print(eval(b))
```

Now if this works for x = 4-r2 it should still work for x = r4+6. Let's test case :

- loop will start as r is in x
- r is at 0 so i = 1
- 1<3 and x[1] = 4 which is a digit so loop continues
- b = x.replace(x[0:1],str(results[int(x[1:1]))])) so b = x with r4 replaced with results[4]
- so b = results[4]+6 hence eval(b) will work.
- I'll test in vscode

Things are working !

Few problems : when I type history I get an error because history has an r in it so the next codeblock runs. So I changed history to timeline.

Another thing is when I type in things like timeline, r1 , r2, r4+4 , they are getting added in the results too , I don't want the timeline command to be added in results so I will get rid of that, I added an if condition before timeline.append(x) that if x is not equal to timeline then only append.

```
y = "stop"
z = "timeline"
results = []
timeline = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
```



```

- stop -> ends program
- rn -> prints n'th result
- timeline -> Prints calculation history"")
while True:
    x = input("Enter to calculate")
    if x!=z:
        timeline.append(x)
    if x == y:
        break
    if x == z:
        print(timeline)
    if "r" in x:
        i = x.index('r')+1
        while i<len(x) and x[i].isdigit():
            i+=1
        b =
x.replace(x[x.index('r'):i],str(results[int(x[x.index('r')+1:i])]))
        print(eval(b))
    else:
        print(eval(x))
        results.append(eval(x))
        print(results)

```

7. I forgot to add input validation. I should make a case at the start of if the input x does not have r in it or is not equal to timeline or is a word then print(enter a valid calculation) or else the remaining code will run. The conditions are confusing to write though.

x should be = timeline, an integer, the letter 'r' along with integers, a combination of integers along with operators ,a combination of the letter 'r' along with operators and integers.

I have two things figured out : that x can be equal to 'stop' or 'timeline'. But how to write a condition for when x = r2-4 or 78-r3, if I just use `if 'r' in x` then even something like x = train will pass. I can see that everytime x has 'r', there is a number after 'r' , like r3, r5 , so r is never on its own. So i need to check if there is a number after 'r' and only then pass is.

For that I need to get the character after r and check if it is a digit.

position of r will be : `x.index('r')`

position of the character after r will be : `x.index('r')+1`

to call this character : `x[x.index('r')+1]`

now checking if this character is a digit : `x[x.index('r')+1].isdigit()`.

If isdigit returns true then the character after r is a digit, otherwise it should be discarded.

The entire condition for calculations to happen should now look like :

```

if x == timeline or x == stop or x[x.index('r')+1].isdigit()
\\ then entire code here

```

```
else
    print("Enter a Valid Calculation")
```

I get an error when I typed just 43 because as conditions were checked left to right, 'r' couldn't be found in last condition `x[x.index('r')+1].isdigit()` so I should make a case of having just digits in it by using `x.isdigit()` right before it.

Then typing in 43 works but when I do 43+90 now it doesn't work because now '43+90' is not a digit. I can make a list of operators and allow x if the operator is in it.

```
operators = ['+', '-', '*', '\\']
if any(i in x for i in operator):
```

but now even something like a+b would get passed.

What if I had a function that could evaluate if each character in my given string meets a condition or not. For something like 4-3, x should have a digit or an operator.

```
all(i.isdigit or i in operators for i in x)
```

is the next required condition.

```
y = "stop"
z = "timeline"
results = []
timeline = []
print("""Welcome to the Calculator!
      Use the following commands to run:
      - type in a calculation + enter -> gives solution & stores result
      - stop -> ends program
      - rn -> prints n'th result
      - timeline -> Prints calculation history""")
while True:
    x = input("Enter to calculate")
    if x==z or x==y or x.isdigit() or all(i.isdigit or i in operators for i in
x) or x[x.index('r')+1].isdigit():

        if x!=z:
            timeline.append(x)
        if x == y:
            break
        if x == z:
            print(timeline)
        if "r" in x:
            i = x.index('r')+1
```

```
        while i<len(x) and x[i].isdigit():
            i+=1
        b =
x.replace(x[x.index('r'):i],str(results[int(x[x.index('r')+1:i]))))
        print(eval(b))
    else:
        print(eval(x))
        results.append(eval(x))
        print(results)
    else:
        print("Enter a Valid Calculation)
```

8. I haven't added feature to do something like $r0+r4$, but I've added the before mentioned features so I should move on to the next project.